

Sandboxing

A hands-on course in building agents by building their harness

Lesson 5

It runs every command, as you

Since Lesson 2, every command the model asks for runs:

- on your machine
- as you, with your files, your network and your credentials
- and nothing asks first

The setup warned you to run it somewhere you can afford to lose. This layer makes that true wherever it runs.

Sandboxing decides where commands run

Instead of your machine, the harness runs them in a box:

- its own filesystem, and a limited view of yours
- no way out to the network
- a fixed share of memory and processor, and a time limit
- thrown away when the run is over

It doesn't decide whether a command *should* run. It decides what a command can reach *when* it runs.

Built on output: same place, a different choice

- Output acts on what the model said: the model asks for a command, output runs it (Lesson 2)
- `subprocess.run` was already a choice of where: it means "right here"
- Sandboxing makes a different choice in the same place
- A production layer adds hardening, not a new primitive

The kernel enforces the walls

- A guardrail is code reading a command and deciding
- A container's limits are enforced by the operating system's kernel
- They hold on whatever runs inside, however the command is spelled
- `rm -rf /` written a hundred different ways can still only see the box's filesystem

Docker is the tool this lesson uses to make the box. It's one of several; the idea is the same in all of them.

The concept: a locked box, and nothing else

`sandboxing.py` has no agent and no model at all. The box (abridged):

```
docker("network", "create", "--internal", net)
up = docker("run", "-d", "--rm", "--name", box, "--network", net,
            "--memory", "256m", ..., "python:3.13-slim", "sleep", "infinity")
if up.returncode: sys.exit(...)
atexit.register(lambda: docker("rm", "-f", box))
```

- 256 MB, at most 64 processes, no capabilities, a read-only root, user `nobody`
- Nothing of yours is mounted; if the box can't start, nothing runs
- Each command goes in with `docker exec`, under a five-second kill

Every command hits a wall

`uv run production/05-sandboxing/sandboxing.py` , the last command a fetch (shortened):

```
$ id; echo hi > /etc/hello
uid=65534(nobody) gid=65534(nogroup) groups=65534(nogroup)
sh: 1: cannot create /etc/hello: Read-only file system
$ sleep 60
(exit 137)
$ python3 -c 'bytearray(1024**3)'
Killed
...
urllib.error.URLError: <urlopen error [Errno -3] Temporary failure in name resolution>
```

With Jev: ask before lending a way out

`jev_sandboxing.py` is `sandboxing.py` plus only Jev's lines. In `run()` (abridged):

```
try: need = jev.system_one({"command": cmd}, {"q": NEEDS}).choices["q"]
except Exception: need = None
lend = bool(need and need.choice == "network"
            and need.confidence >= SURE and read(...) == "y")
if lend: docker("network", "connect", "bridge", box)
done = docker("exec", box, "timeout", "-s", "KILL", "5", "sh", "-c", cmd)
if lend: docker("network", "disconnect", "bridge", box)
```

- `NEEDS` : nothing, network, or outside? Only a sure `network` asks the person
- Anything else, or no answer: the closed box

The same fetch, lent a way out

```
uv run production/05-sandboxing/jev_sandboxing.py ; I answered y
```

(shortened):

```
$ id; echo hi > /etc/hello
[Jev: outside, 1.00]
...
[Jev: nothing, 0.94]
...
[Jev: nothing, 0.95]
...
[Jev: network, 1.00]
lend it the network for this one command? [y/N] y
200
(exit 0)
```

Asking Jev

- A model that writes no text: send it a state and typed questions, get typed answers
- **Choice** picks an option, **noul** is yes-or-no, **score** places it on a scale
- The answer says what; a separate **confidence** says whether to act on it
- About \$0.00002 a question, and about a fifth of a second
- Its own key, `TYPESAFE_API_KEY` ; without it, every lesson carries on as if it never asked

What Jev is weak at, and where it sits

- **It reads literally:** put the boundary cases in the options
- **It doesn't count**, and **indirection** costs it (`npm test` needs the network only if the tests do)
- **Text can steer it:** a command written to look harmless can be judged harmless
- Asking Jev is **model interface**; acting on the answer belongs to the layer's own primitive
- Here that's **output**: its answer only decides whether you're asked

quark: the box, made for an agent

Lesson 4's `quark.py` plus 29 lines, 268 in all. `sandbox()` in the output section (abridged):

```
def sandbox():
    where = os.getcwd()
    subprocess.run(["docker", "network", "create", "--internal", net], ...)
    up = subprocess.run(["docker", "run", ..., "--network", net, ...,
                        "--user", f"{os.getuid()}:{os.getgid()}",
                        "-v", f"{where}:{where}", "-w", where, IMAGE, "sleep", "infinity"], ...)
    if up.returncode: sys.exit(...)
```

- Runs as *you*, with the folder `quark` started in mounted at the same path
- 512 MB, 128 processes, 30 seconds a command; removed at exit, then its network

quark: asking Jev, and lending the way out

In the model interface section, `jev`, `SURE` and `ask()` (abridged):

```
def ask(state, question):  
    try: return jev.system_one(state, {"q": question}).model_dump()["answers"]["q"]  
    except Exception: return None
```

And in output, `lend()` and `bridge()` (abridged):

```
def lend(cmd):  
    need = ask({"command": cmd}, NEEDS)  
    return bool(need and need["choice"] == "network"  
                and need["confidence"] >= SURE and read(...) == "y")  
  
def bridge(on):  
    subprocess.run(["docker", "network",  
                   "connect" if on else "disconnect", "bridge", box], ...)
```

One line moves the command into the box

`subprocess.run(cmd, shell=True)` became this (abridged):

```
lent = lend(block.input["cmd"])
if lent: bridge(True)
done = subprocess.run(["docker", "exec", box, "timeout", "-s", "KILL",
                        str(TIMEOUT), "sh", "-c", block.input["cmd"]], ...)
if done.returncode == 137:
    done.stdout += f"\n(killed: ran over {TIMEOUT} seconds or ...)"
if lent: bridge(False)
```

- Same shell, same merged output and exit code, but in the box under a 30-second limit
- Exit 137: a time-out *or* out of memory; quark tells the model both

Everything else is Lesson 4's, on purpose

- The prompt still tells the model bash reaches "the whole system"; inside the box, that's true
- Nothing tells the model about the box, or when it was lent the network
- That would be a decision about context, and this layer doesn't make it
- But `mechanics()` shows the model its own file, so it can read `sandbox()` (Lesson 4's doing)

Run it: one write works, three hit walls

In a scratch folder: write `hello.txt`, then try `/etc`, `example.com` and the API keys (shortened):

```
hi
---
sh: 1: cannot create /etc/testfile: Read-only file system
etc exit: 2
---
urllib.error.URLError: <urlopen error [Errno -3] Temporary failure in name resolution>
---
keys-no
---
uid=0(root) gid=0(root) groups=0(root)
```

Nobody was asked about the network

- Part of that one command tried to reach example.com
- Jev judges the command as a whole, and this one mostly writes and reads files
- Asked again separately: `network`, with a confidence of 0.44
- Under 0.9, so no question, and the command ran in the closed box
- That's the direction it's meant to fail in

A way out for one command

Fetch a page from an Ubuntu mirror, then check again. I answered `y`, then `n` (shortened):

```
"` needs the network: allow it for this one command? [y/N] y
Index of /ubuntu
...
"` needs the network: allow it for this one command? [y/N] n
Index of /ubuntu
...
urllib.error.URLError: <urlopen error [Errno -3] Temporary failure in name resolution>
```

The first command must have been given temporary network access, and the second wasn't. So the failure says nothing about whether `archive.ubuntu.com` is up.

A killed command doesn't take the run with it

A copy with `TIMEOUT` set to 5 (shortened):

```
$ sleep 60

(killed: ran over 5 seconds or out of memory)
$ python3 -c 'x = bytearray(2*1024**3); x[:] = b"a"*len(x)'
Killed

(killed: ran over 5 seconds or out of memory)
$ ls
hello.txt
notes.txt
title.txt
```

Other things we could do

- **The box:** a container, gVisor, a Firecracker micro-VM, a full VM, or the OS's own process sandboxing
- **What it sees and reaches:** a read-only mount or a copy; an allow list, or a proxy that adds credentials
- **What it gets, and how long it lives:** disk, output, time per run; one box per command, run or session
- **How results get out:** a shared folder, a person says yes, or a patch or pull request
- **A product on its own:** hosted sandboxes like E2B, Modal and Daytona

Ideas that close gaps quark leaves open

- **Work on a copy:** pack the folder in, leave out `.env` and `.git`, and let a person pick what comes back out
- **No disk to fill:** only small in-memory areas with sizes
- **Run as** `nobody`, who owns nothing even inside the box
- **Cap the output inside the box** with `head -c`, so a command that prints forever can't fill the harness's memory
- **Lend less than everything:** an allow list or a proxy, not the whole network

The rule: run commands where they can't do lasting damage

- Make the box before the loop
- Aim the tool at it in place of `subprocess.run` on the host
- Limit what it can see, reach, use and change with the operating system, not with checks on the text of a command
- Throw it away when the run ends

What sandboxing never does

It sits inside output, changing where a tool runs, and leaves the other primitives alone:

- **Model interface:** still talks to the model from outside the box; asking Jev only decides whether you're asked
- **Control flow:** the same loop
- **Input:** still brings back what a command printed; your `y` or `n` comes through `read()`
- **Context:** untouched; nothing about the box goes into the request

What's missing: deciding whether it runs

- The box limits what a command can reach, not whether it runs
- Inside it everything still runs unasked, including `rm -rf` on the mounted folder: your real project
- The only question quark asks is about the network
- Nothing lets you say no to any other command, or break in while it works
- Nothing stops a run that loops forever, spending your money one call at a time

Next: Guardrails

Lesson 6